Міністерство освіти і науки України
Національний університет "Львівська політехніка"
Кафедра ЕОМ



Звіт

з лабораторної роботи № 2

з дисципліни: «АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ СИСТЕМ»

на тему «Комунікація клієнта та сервера »

Варіант №21

Виконала:
ст. гр. КІ-404
Хилинська Л. В.
Прийняв:
Федак П. Р

Львів – 2024

Порядок виконання роботи:

1. Створити репозиторій на GitHub з назвою "csad", де основна гілка - develop. YY1 – рік початку навчання YY2 – рік закінчення навчання Наприклад: 2019-2020 -> 1920 Повний приклад: csad1920ki47spitzeras03
2. Додати доступ для автора.
3. Створити гілку feature/develop/. Наприклад, feature/develop/task1.
4. Створити файл README, де вказати:

- деталі про репозиторій;
- деталі завдання;
- номер студента та інформацію.
- додати інформацію про технології, мову програмування та обладнання, яке буде використовуватися в наступних завданнях.

5. Створити GIT TAG:

   __WW · YY – поточний рік; WW – робочий тиждень; D – номер дня тижня.

6. Створити pull request з назвою task1 та надати викладача як рецензента.
7. Після затвердження рецензента – виконати злиття запиту в гілку develop. Критерії прийняття - Посилання на створений репозиторій з гілкою "develop" як основною та файлом README.MD.

Варіант завдання:

Student number	Game	config format
21	tik-tac-toe 3x3	INI

Теоретичний матеріал

UART (Universal Asynchronous Receiver-Transmitter) — це апаратний модуль або інтегральна схема, що використовується для реалізації асинхронного послідовного обміну даними між двома пристроями.

Основні характеристики UART:

1. Функціонал:
   - Приймає паралельні дані від мікропроцесора і перетворює їх у послідовний потік для передачі.
   - Перетворює отриманий послідовний потік даних назад у паралельний для використання приймачем.
2. Принцип роботи:
   UART працює без синхронізації з зовнішнім тактовим сигналом (асинхронно), замість цього передача даних регулюється стартовим і стоповим бітом.
   - Стартовий біт — сигналізує про початок передачі.
   - Стоповий біт — вказує на завершення передачі.
   - Біти даних — передають корисну інформацію (зазвичай від 5 до 9 бітів).
   - Біт парності (опційно) — використовується для перевірки помилок.
3. Швидкість передачі:
   Швидкість UART визначається бод-рейтом — кількістю бітів, що передаються за секунду (наприклад, 9600 біт/с). Важливо, щоб бод-рейт передавача і приймача збігалися.

CMake — це кросплатформовий інструмент автоматизації складання програмного забезпечення, який використовується для створення файлів конфігурації збірки. Він дозволяє генерувати файли збірки для різних систем і

компіляторів (наприклад, Makefile, Ninja, Visual Studio Projects) та забезпечує гнучкий підхід до організації процесу збірки.

Основні компоненти CMake

1. CMakeLists.txt: Центральний конфігураційний файл, який визначає структуру проекту, джерела, цілі, залежності, опції компіляції та тестування.
2. Ціль (Target):Основна одиниця збірки. Ціллю може бути виконуваний файл, бібліотека або тестовий модуль.
3. Генератори:CMake використовує генератори для створення вихідних файлів збірки (наприклад, Makefile або проектів для IDE).

CI/CD (Continuous Integration / Continuous Delivery) — це набір практик і інструментів, які використовуються для автоматизації процесу розробки, тестування, інтеграції та доставки програмного забезпечення. Вони допомагають зменшити ризики, підвищити якість продукту та скоротити час виходу оновлень на ринок.

GitHub Actions — це вбудована система автоматизації у GitHub, яка дозволяє створювати, налаштовувати та виконувати сценарії (workflow) безпосередньо у вашому репозиторії. Вона підтримує CI/CD, автоматичне тестування, розгортання, перевірку коду, повідомлення та багато іншого.

Основні поняття GitHub Actions

- Workflow: Набір завдань, що описуються у YAML-файлі та виконуються у відповідь на певну подію.
- Job: Завдання, що виконуються на окремих віртуальних машинах. Кілька job'ів можна виконувати паралельно або послідовно.
- Step: Окремий крок у job'і. Це може бути команда, скрипт або дія.
- Action: Готовий крок (або набір кроків), який можна використовувати повторно.

Порядок виконання роботи:

1. Створила просту схему комунікації ПЗ (клієнт) <-> UART <-> Апаратне забезпечення (сервер).

```
lili@lili-VirtualBox task tree
.
├── clientSide
│   ├── CMakeLists.txt
│   ├── include
│   │   └── constants.h
│   └── src
│       └── clientSide.cpp
├── README.md
├── reports
│   └── АПКС_КІ-404_Хилинська_task1.docx
└── serverSide
    ├── CMakeLists.txt
    ├── include
    │   └── constants.h
    └── serverSide.ino
```

2. Клієнт повинен відправити повідомлення серверу. Сервер має модифікувати повідомлення і надіслати його назад клієнту.

```
lili@lili-VirtualBox build ./client
Opening port '/dev/ttyUSB0'
Waiting for Arduino to be ready...
Received from Arduino: Arduino ready!

Enter message to send to Arduino: this is the test :P
Message sent: this is the test :P
Response from Arduino: THIS IS THE TEST :P

Enter message to send to Arduino:
```

3. Створила YML файл з наступними функціями: а. збірка всіх бінарних файлів;

```yaml
name: CI Pipeline

on:
  push:
    branches:
      - '*'
  pull_request:
    branches:
      - '*'

jobs:
  build-client:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Install dependencies
        run: sudo apt-get update && sudo apt-get install -y libserialport-dev

      - name: Setup CMake
        uses: lukka/get-cmake@v3.23.0

      - name: Create build directory
        run: mkdir -p build
      - name: Configure CMake
        run: |
          cd build
          cmake ../clientSide

      - name: Build client using CMake
        run: |
          cd build
          cmake --build .

      #- name: Run tests
      #  run: |
      #    TBD

  build-server:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Arduino CLI
        uses: arduino/setup-arduino-cli@v1
        with:
          version: '0.19.2'

      - name: Install Arduino core for AVR boards
        run: arduino-cli core install arduino:avr

      - name: Build Arduino sketch (Server)
        run: arduino-cli compile --fqbn arduino:avr:uno serverSide/serverSide.ino

      #dummy
      - name: Upload Arduino Build Artifact
        uses: actions/upload-artifact@v4
        with:
          name: arduino-build
          path: serverSide/build/

      #- name: Run tests
      #  run: |
      #    TBD
```

Висновок:У ході виконання завдання було розроблено схему взаємодії між програмним забезпеченням (клієнтом) і апаратною частиною (сервером) за допомогою інтерфейсу UART. Клієнт ініціює передачу даних до сервера, який обробляє отриману інформацію, вносить необхідні зміни та надсилає їх назад клієнту, забезпечуючи двосторонній зв'язок.

Окрім цього, було впроваджено автоматизовану систему для збірки та тестування програмного коду. Вона включає YAML-конфігурацію для CI/CD-процесів, яка автоматизує збірку необхідних бінарних файлів, організацію скриптів у відповідній структурі, а також створення артефактів із зібраними файлами та результатами тестів. Такий підхід забезпечує стабільність і надійність розробки завдяки безперервній інтеграції та доставці.